Pontifícia Universidade Católica de Minas Gerais
Teste de Software

Refatoração de Testes e Detecção de Test Smells

Repositório: github.com/bacelete/test-smelly

Aluno: Arthur Henrique Teixeira e Silva Bacelete

Matrícula: 859123

Belo Horizonte, outubro de 2026

1. Análise de Smells

A suíte original (`test/userService.smelly.test.js`) passa por completo, mas contém vários *test smells*. Destaco três.

1.1 Lógica Condicional no Teste (Conditional Test Logic)

Onde: teste *"deve desativar usuários se eles não forem administradores"*. O teste percorre uma lista com `for` e decide com `if/else` quais asserções executar, misturando dois cenários (usuário comum e administrador) num único teste.

Por que é um mau cheiro e qual o risco: o teste deixa de ser uma sequência linear de passos e, para entender o que ele verifica, é preciso "executá-lo de cabeça". Se a lista mudar ou o `if` tomar sempre o mesmo caminho, parte das asserções nunca roda e o teste continua verde (falso positivo). Quando ele falha, o nome não diz qual cenário quebrou.

1.2 Tratamento de Exceção Silencioso (Exception Handling)

Onde: teste *"deve falhar ao criar usuário menor de idade"*. O `expect` está dentro de um `catch`: se `createUser` **não** lançar exceção, nenhuma asserção roda e o Jest aprova o teste.

Por que é um mau cheiro e qual o risco: o teste verifica a mensagem do erro *quando* ele acontece, mas não verifica *que* ele acontece. É o smell mais perigoso da suíte. Para comprovar, removi temporariamente a validação `idade < 18` do serviço: a suíte original continuou 100% verde, enquanto a suíte refatorada falhou imediatamente. É o mesmo "mutante sobrevivente" do trabalho anterior de teste de mutação.

1.3 Teste Frágil / Igualdade Sensível (Fragile Test)

Onde: teste *"deve gerar um relatório de usuários formatado"*. O teste monta a linha esperada caractere por caractere (`ID: ..., Nome: Alice, Status: ativo\n`) e exige um cabeçalho exato.

Por que é um mau cheiro e qual o risco: o teste fica acoplado ao *formato* da saída, e não ao *conteúdo*. O próprio código avisa que "o formato do relatório pode mudar no futuro". Qualquer mudança cosmética quebra o teste sem que exista bug, o que gera manutenção desnecessária e faz a equipe perder a confiança na suíte.

Outros smells encontrados: *Eager Test* ("criar e buscar" testa duas operações, com dois "Act"), *teste desabilitado/vazio* (`test.skip` com apenas um TODO) e *fixture geral* (objeto `dadosUsuarioPadrao` global usado por um único teste).

2. Processo de Refatoração

Escolhi o teste de desativação de usuários, que reúne laço, lógica condicional e dois cenários num só. Ao lado, o "antes" e o "depois".

Antes: userService.smelly.test.js

```
test('deve desativar usuários se eles não
      forem administradores', () => {
  const usuarioComum = userService.createUser(
    'Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser(
    'Admin', 'admin@teste.com', 40, true);

  const todosOsUsuarios =
    [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
    const resultado =
      userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado =
        userService.getUserById(user.id);
      expect(usuarioAtualizado.status)
        .toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois: userService.clean.test.js

```
describe('deactivateUser', () => {
  test('desativa um usuário comum', () => {
    // Arrange
    const usuarioComum = criarUsuarioComum();
    // Act
    const resultado = userService
      .deactivateUser(usuarioComum.id);
    // Assert
    expect(resultado).toBe(true);
    expect(userService.getUserById(
      usuarioComum.id).status).toBe('inativo');
  });

  test('não desativa um usuário administrador',
    () => {
    // Arrange
    const usuarioAdmin = criarUsuarioAdmin();
    // Act
    const resultado = userService
      .deactivateUser(usuarioAdmin.id);
    // Assert
    expect(resultado).toBe(false);
    expect(userService.getUserById(
      usuarioAdmin.id).status).toBe('ativo');
  });

  test('retorna false quando o usuário não
        existe', () => { /* ... */ });
});
```

Decisões tomadas:

- **Um cenário por teste.** O teste foi dividido em três (comum, administrador e inexistente). Quando um falha, o nome já diz qual regra quebrou.
- **Sem `for` e sem `if`.** Todas as asserções sempre executam, o que elimina os erros `jest/no-conditional-expect`.
- **Padrão AAA explícito**, com um único *Act* por teste.
- **Asserção mais forte para o admin.** Além do retorno `false`, o teste confirma que o status *continua* `ativo`, ou seja, verifica o efeito real da regra de negócio.
- **Funções auxiliares** (`criarUsuarioComum`, `criarUsuarioAdmin`) substituem o fixture global e deixam o *Arrange* curto.

Como os demais smells foram corrigidos:

Smell original	Correção na suíte limpa
`try/catch` com `expect` condicional	`expect(() => ...).toThrow('mensagem')`, que falha se a exceção **não** for lançada
Relatório com formato exato	Verifica que o relatório **contém** id, nome e status de cada usuário, sem depender da formatação
Eager Test (criar + buscar)	Testes separados em `describe('createUser')` e `describe('getUserById')`
`test.skip` vazio	Implementado: "informa que não há usuários quando o cadastro está vazio"
Casos não cobertos	Novos testes para idade limite (18), campos obrigatórios ausentes (`test.each`), ids únicos e id inexistente

3. Relatório da Ferramenta (ESLint)

Usei a configuração `.eslintrc.json` do enunciado com `eslint@8` e `eslint-plugin-jest@27`. Escolhi a versão 8 porque o ESLint 9 não lê mais o formato `.eslintrc.json` e exige Node 18 ou superior. Resultado da primeira execução de `npx eslint .`:

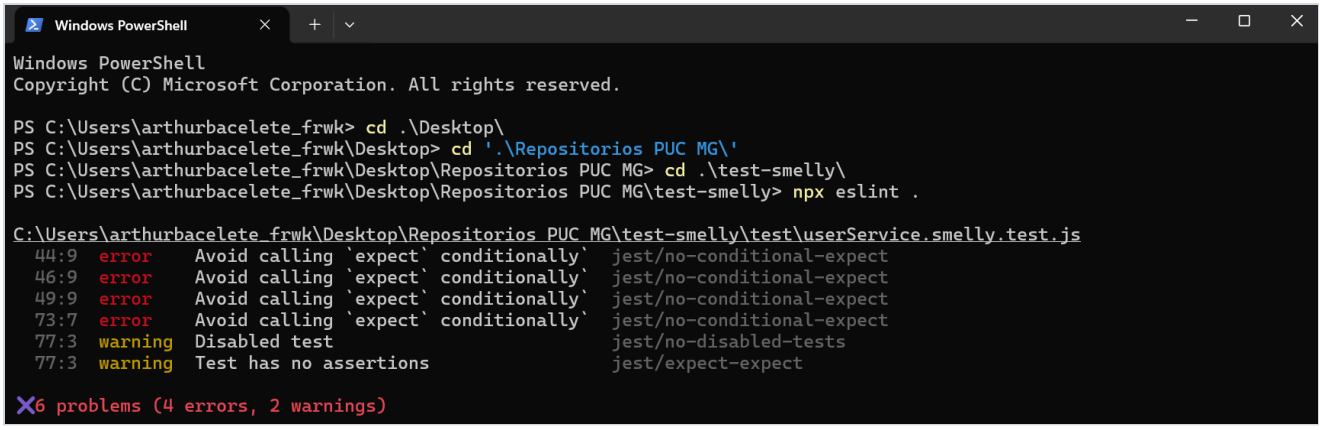


Figura 1: primeira execução do ESLint sobre a suíte original (4 erros e 2 avisos).

Como a ferramenta automatizou a detecção: as linhas 44, 46 e 49 correspondem ao `if/else` dentro do `for` (seção 1.1). A linha 73 é o `expect` dentro do `catch` (seção 1.2). A linha 77 aponta o `test.skip` vazio de duas formas: teste desabilitado e teste sem asserções.

Comparação com a análise manual: o ESLint encontrou sozinho 2 dos 3 smells principais, além do teste desabilitado. Ele **não** detectou o Teste Frágil nem o Eager Test, porque são problemas de *intenção* e de projeto do teste, que uma regra sintática não consegue julgar. Ferramenta e revisão humana se complementam.

Validação final: `npx eslint test/userService.clean.test.js` não reporta nenhum problema, e `npm test` passa nas duas suítes (19 testes aprovados e 1 ignorado, que é o `skip` do arquivo original, mantido intocado).

4. Conclusão

A suíte original passava por inteiro e, ainda assim, não protegia o sistema: sem a validação de idade mínima, ela continuou verde, enquanto a suíte refatorada acusou o defeito na hora. Isso confirma o que o teste de mutação já tinha mostrado: um teste que passa não é necessariamente um teste que verifica algo.

Testes limpos (um comportamento por teste, padrão AAA, sem lógica condicional e verificando comportamento em vez de formato) **detectam bugs de fato** e **documentam as regras de negócio**: só pelos nomes dos testes de `deactivateUser` já se entende que administradores não podem ser desativados. Ferramentas de análise estática como o ESLint com `eslint-plugin-jest` tornam essa disciplina sustentável. No editor ou na esteira de CI, elas barram automaticamente os smells mecânicos antes do *code review*, e a revisão humana pode se concentrar no que a ferramenta não enxerga.